Software Engineering

CSC648/848

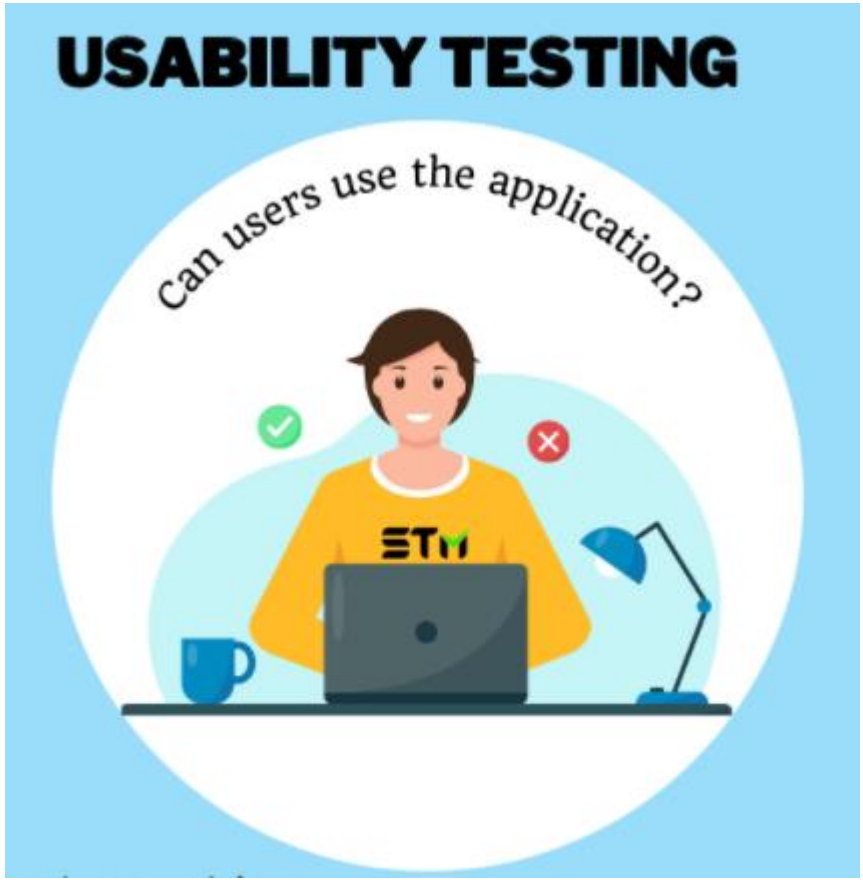
Class Plan

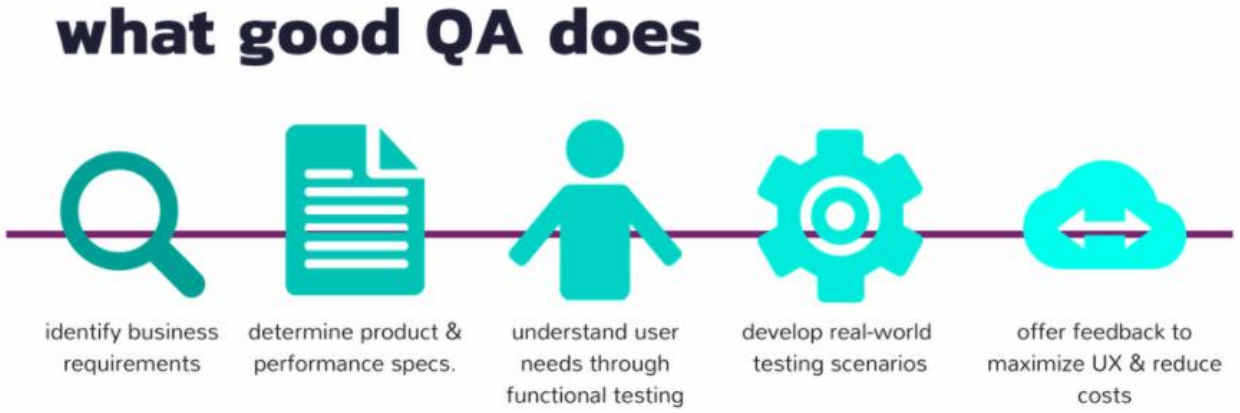
- Team 6 Takeaways Week 12 SE Methodology and Process Part 2
- M4 Feedback. Preparing for the Future. Resume and LinkedIn Best Practices & What makes a great Software Engineer? SE Tooling.
- Team Collaboration

M4 Feedback

There are several basic components present in all SW Engineering methodologies:

1. *SW Requirements and Specification*: functionality, constraints, look and feel, GUI, performance
2. *SW Design and Implementation*
3. *SW Validation*: formal, test plans, QA
4. *SW Maintenance and Evolution*: ongoing changes (and fixes) to meet customer needs





Images Source: Internet (Software Testing Material & Qualitylogic.com)

Henry
VILLAR

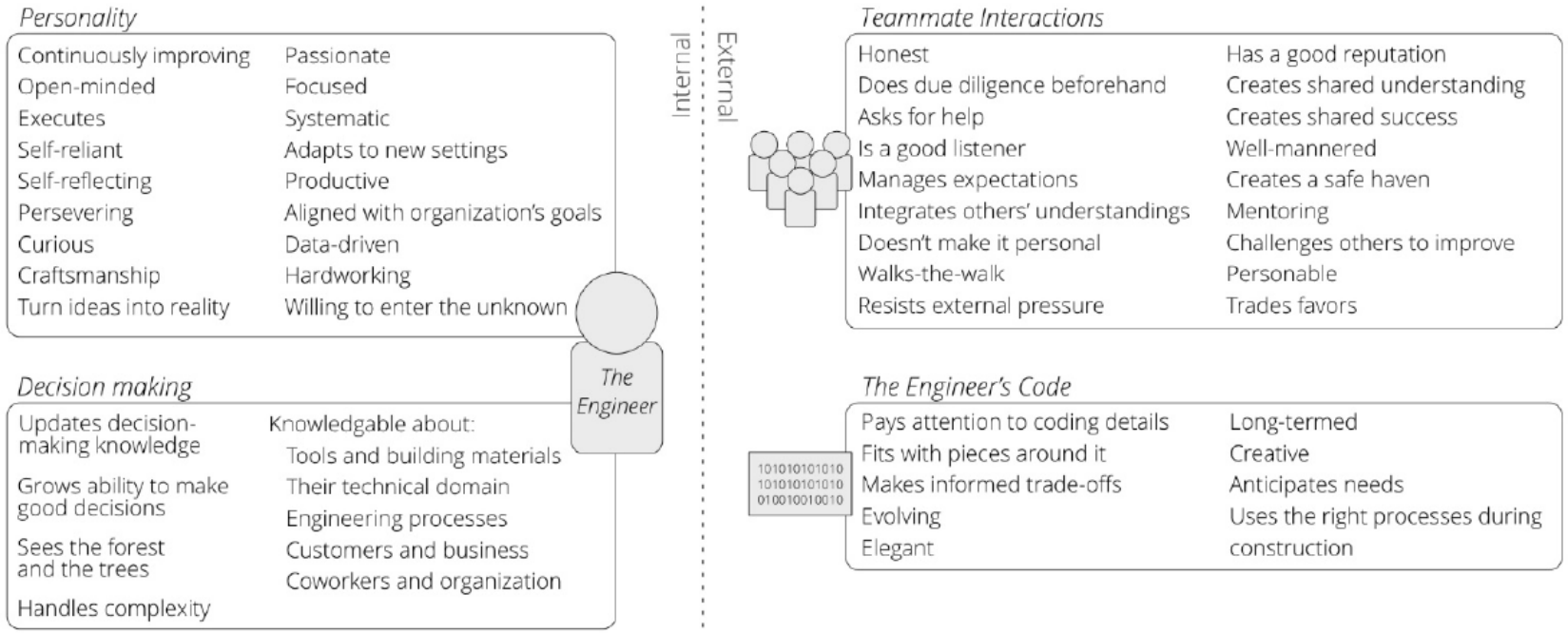
SE REVIEW

Preparation for the Future

What makes a great Software Engineer?

What makes a great software engineer?

http://dl.acm.org/citation.cfm?id=2818839



This work was supported in part by Microsoft, Google, and the National Science Foundation (NSF) under Grants CCF-0952733, CNS-1240786, and IIS-1314399.

What makes a great Software Engineer?

- Take Away #1: The ability to learn is more important than any individual technical skill
- Take Away #2: Making good decisions is rarely discussed in the software engineering literature, but it is critical to being a great software engineer
- Take Away #3: Software engineering is a sociotechnical undertaking
- Take Away #4: Delivering the code is often insufficient; complex contextual technical considerations abound.

Resume Best Practices

Software Engineer Resume — Best Practices

Recommendations for Creating a Software Engineering Resume

Steve Jones Follow

Dec 18, 2015 · 6 min read

https://medium.com/@steve_jones/7-software-engineer-resume-best-practices-7598636bdf3f

LinkedIn Best Practices

Software Engineers: What makes your LinkedIn profile 10/10?

Published on August 23, 2017

Daan Smit
Senior IT Recruiter open for a new opportunity: +31650533812

1 article + Follow

You're ready for your next challenge. You changed your LinkedIn profile to let recruiters know you're open. Now it's time to get noticed. But what makes your profile pop out?

As a TomTom talent sourcer, I focus on Software Engineers and have seen thousands of profiles. Here are some quick and easy tips and tricks for making your profile catch and keep a recruiter's eye.

The Basics:

Summary: I strongly prefer a short description, for example: "C++ Rocks!" or "Java Geek Exploring Big Data". By describing what you're interested in (Machine Learning / Algorithm and Math / Clean Coding and Scrum), it generates a quick and clear picture of your interests.

Experience: I look for software professionals (engineers, developers, architects, scrum

https://www.linkedin.com/pulse/software-engineers-what-makes-your-linkedin-profile-1010-daan-smit/

Know before you…Interview

- Listen "**Gayle Laakmann McDowell Talks about 'Interview Like Google (or not)-But Better!'**" video
- Be familiar with some of the testing tools some companies use:
  - https://karat.com/candidates/how-it-works/
  - Karat Breakdown in terms of time. It is exactly 60 minutes (they will cut you off)
    - SWE Knowledge Questions (10 Minutes)
    - Coding Exercise (45 minutes)

Emails and Prep

SDET

Hello <name> ,

- Great News! We would like to move forward in the process and have you speak to two of our engineers - <engineer 1 & engineer 2> via zoom for up to 90 mins.
- This will be a **technical interview that involves a pair coding session** to develop an actual code of automated tests for a simple set of APIs/front-end application. To prepare, please set up your environment - IDE and starter testing framework of choice (e.g. Selenium, REST Assured, etc), and the other tools you find necessary. Please, prepare it before an actual call as the time for an interview will be limited. You will also be asked to share your screen via zoom so that the interviewer can review the code as you build in your choice of IDE.

Front end

Hello <name> ,

- Great News! We would like to move forward in the process and have you speak to two of our engineers - <engineer 1 & engineer 2> via zoom for up to 90 mins.
- This will be a **technical interview that involves a pair coding session** to develop a simple two-page application. To prepare, please set up your environment (IDE and starter framework of choice (e.g. React, Angular, etc), and the other tools you find necessary. Please, prepare before the interview since time will be limited. You will also be asked to share your screen via zoom so that the interviewer can review the code as you build in your choice of IDE.

Back-end

Hello <name> ,

- Great News! We would like to move forward in the process and have you speak to two of our engineers - <engineer 1 & engineer 2> via zoom for up to 90 mins.
- This will be a **technical interview that involves a pair coding session** to develop an actual code to build a small workable server component. To successfully pass this interview, you will need to have a fully operational development environment that includes IDE of your choice and the other tools you find necessary. Please, prepare it before an actual call as the time for an interview will be limited. You will also be asked to share your screen via zoom so that the interviewer can review the code as you build in the setup, which you feel most comfortable in.

Questions (Modern JavaScript)

Question 1 - Defaulting / Destructuring

This is a small code review question.

Prompt 1

This function is written in an older style of JavaScript and we want to modernize it. What changes would you make to the function to take advantage of newer features such as default parameters or destructuring parameters? You can also change anything else you want to be more modern, and change the function signature if you want. Just be sure that all functional requirements are still met!

```
function greet(greeting, name) {
  greeting = greeting || "Hello";
  name = name || "World";
  return greeting + ", " + name + "!";
}

// functional requirements / expected outputs:
//
// pass in nothing:                              Hello, World!
// pass in only a greeting 'Howdy':              Howdy, World!
// pass in both a greeting 'Howdy' and name 'Karison":  Howdy, Karison!
// pass in only a name 'Karison':                Hello, Karison!
```

Scoring

select all valid of:

- correctly changed the function to default parameters or destructuring
- changed string concatenation to string interpolation
- if using default parameters, understood the difference between null and undefined to skip the first parameter, or if using destructuring knew how to handle the case of greet() vs. greet({})

Question 2 - This and That

Prompt 1

Why would this code cause the two TypeError marked by the comments?

```
"use strict";

function MyObj(meaning) {
  this.value = meaning;
  this.title = "Meaning of life";

  this.report = function () {
    return `${this.title}: ${this.value}`;
  };
}

let a = new MyObj(42);
console.log(a.report()); // Meaning of life: 42

let aReport = a.report;
console.log(aReport()); // TypeError: Cannot read property 'title' of
undefined

let b = MyObj(42); // TypeError: Cannot set property 'value' of undefined
console.log(b.report());
```

Scoring

select all valid of:

- correctly identified the problem of this not working for a function when an object prefix is not used at the call site.
- correctly identified that the missing new would cause the function to not work as-is as a constructor without change.
- correctly identified the solution of changing the function report() to an arrow function so that it captures the this of its object instance.

Plans for next 3 classes

- Week of Nov 25-29 – Fall Recess
- W14 – Dec 3rd 2024: Prep for Demo and Final Exam | Team Collaboration for M5
- W15 – Dec 10th 2024 | **Milestone 5 Demo and Documentation**
- W16 – Dec 17th 2024 **FINAL EXAM** @6PM-7:30PM (1.5h)

Milestone 5

- Final Demo and Documents for Grading (Due date: December 10th – by 5PM). NOT EXCEPTIONS!
  - Delivery 1: Demo the Application (15 min).
  - Delivery 2: Folder with complete project documentation – "M5 folder"
    - e-mail subject line: Must be "CSC648-848 Fall 2024 Section 2 Milestone5 Folder Team N" in the subject line (N is a team number).
    - e-mail body contains direct link to Milestone 5 folder in team github

    1. Cover Page
    2. Product Summary
    3. Milestone Documents (M1-M4)
    4. Screenshots of actual final product as shown in the demo
    5. Screenshots of key DB tables
    6. Google analytics plot for your WWW site
    7. Screenshots of your task management system (like Trello) showing a snapshot of your project Management
    8. Team Members contributions
    9. Post analysis – lessons learned

- Schedule (Tuesday Dec 10th):
  - 06:10-06:25PM – Team #2
  - 06:30-06:45PM – Team #5
  - 06:50-07:05PM – Team #1
  - 07:10-07:25PM – Team #6
  - 07:30-07:45PM – Team #4
  - 07:50-08:05PM – Team #3

Individual Class Reflection Paper

1. Analysis of the entire Software Engineering class
2. Experience working with the team project
3. Innovation Space
4. Resume
5. Preferred Job and Plan to enter the High Tech and Job Market as Computer Scientist or any other position.

Due Date: Dec 10th **by 5PM** NOT EXCEPTIONS!

Rubrics for team project grading

Measured Team Outcomes for SE *process*	**Measured Team Outcomes for SE *product***
1. Fraction of the team participating at the meetings with the instructor 2. Quality and timing of follow-up on outstanding issues 3. Ability to deal with feedback constructively 4. Producing the non-SW and SW deliverables on time; following submission guidelines 5. Quality and completeness of non-SW deliverables (requirements, use cases, UI mockups, design documents, test plans etc.) as in M5 folder 6. Number of teamwork issues that instructor had to deal with (whether resolved or not) 7. Ability to apply best SE process and teamwork practices 8. Adherence to continuous and collaborative development practices 9. Ability to effectively use the SE tools (e.g. collaborative tools, version control, issue trackers)	1. Correctness and reliability of operation 2. Functionality of the product vs. desired requirements and use cases 3. Ease of use, user interface 4. Architecture of the developed system 5. Database design 6. Basic security, field validation 7. Performance 8. Code quality and “smell” (coding principles, style, documentation) 9. Proper usage of collaborative tools e.g. repository 10. Presentation style and effectiveness of final product demonstration (content, delivery, adherence to time, dealing with Q/A)
25 %	25 %

Thank You